One Codebase, Every Mobile Platform

A Flutter experiment sharing a single codebase across mobile platforms.

What We'll Cover

1. The concept
2. Approaches and tools
3. The tiny project
4. Outcomes

One codebase ships everywhere

One source, four platforms, no duplicates

1

shared codebase replaces separate Swift and Kotlin projects.

4

targets compiled from that source: iOS, Android, web, desktop.

1

place to fix a bug instead of patching it per platform.

Cross-platform trades peak speed

One shared codebase, near-native speed

Cross-platform

1 codebase

Ships faster at near-native performance.

tradeoff

Native

Full UI access

One project per platform, top performance.

Three frameworks, three models

Flutter, React Native, and .NET MAUI differ mainly in language and rendering model.

Flutter



Dart, own-rendered widgets via Skia

React Native



JavaScript, maps to native widgets

.NET MAUI



C#, renders to native controls

Choice hinges on team language and whether own-rendered or native UI fits the project.

A tiny note-taking app



Create

Add a new note



Edit

Change note text



List

See all saved notes



Delete

Remove a note

Widgets over a storage layer



A widget tree reads and writes through one service

List and editor screens form the UI layer.
A storage service wraps the device's local key-value store.
Edits call save, and the list rebuilds from stored data.



Two screens, one save flow

The list screen shows every saved note. The editor writes changes straight to local storage, then the list rebuilds to show them.

Note list

All notes, tap to open

Editor

Type and save text

Save to storage

Persist, then rebuild list

Empty list on load

Storage read raced ahead of data

Fixed by awaiting the storage call and showing a loading state until it resolved.

Layout off per platform

Same screen felt cramped, then loose

Fixed with responsive padding tied to screen width across devices.

One codebase, several platforms

One source, every platform

The finished app runs from one source on more than one platform, with the same four features working on each.

> **Write once ran everywhere, exactly as claimed on slide one. Four features, multiple platforms, one source.**

The build confirmed the earlier tradeoff: a little peak performance and the async and layout costs traded for a far faster ship than separate native projects. Planning storage timing and responsive layout early paid off across every platform.

One codebase proved it could ship everywhere

A single Flutter note app ran across platforms, resolving the tradeoffs it set out to test.

Three frameworks, one goal

Flutter, React Native, and .NET MAUI each write app logic once for iOS, Android, web, and desktop.

- Flutter: Dart, own-rendered widgets
- React Native: JavaScript, native widgets
- .NET MAUI: C#, native controls

A minimal note app proved the model

A widget tree and a local storage service handled create, edit, list, and delete for on-device notes.

- List screen shows all notes, tap to open
- Editor writes changes straight to storage
- Save persists, then rebuilds the list

Two bugs, two fixes in code

Async storage timing and platform layout gaps surfaced once the app ran on real devices.

- Empty list on load: await storage, show loading state
- Layout off per platform: responsive padding by screen width

Thank You